Context Engineering 横纵分析报告

> 2025 年诞生的新概念，还是 LLM 应用工程化绕不开的终局？从"怎么写指令"到"给模型看什么"的范式转移。

一、一句话定义

Context Engineering（上下文工程）是**设计、编排和管理 LLM 上下文窗口内全部信息的系统工程**。它的核心矛盾在于：模型能处理的上下文长度在迅速增长（4K → 2M），但"放得下"不等于"用得好"——信息的组织方式、优先级、新鲜度、缓存策略，这些系统工程问题正在取代"怎么写 prompt"成为 AI 工程的核心挑战。

> 🎯 **读完这篇你能**：理解上下文窗口"放得下不等于用得好"的核心洞察，能设计合理的上下文编排策略（信息排序、缓存复用、注意力预算管理）来提升 LLM 应用的准确率和性价比。

二、技术背景

核心矛盾：上下文窗口在变大，但"塞满"不是答案

大语言模型的上下文窗口正在以惊人的速度增长——从 2022 年的 4K token 到 2026 年的 2M token，扩大了 500 倍。但工程实践反复证明几个反直觉的事实：

1. **更长 ≠ 更好**："Lost in the Middle"（Liu et al., 2023）揭示——信息放在 prompt 中间位置时，即使内容相关，模型也倾向于忽略它。
2. **注意力是稀缺资源**：Transformer 的 pairwise attention 意味着 token 越多，每个 token 分到的注意力越稀疏。Anthropic 在 2025 年明确提出"Attention Budget"理论——每增加一个 token 都在消耗这个预算。
3. **Context Rot**：随着上下文长度增长，模型准确回忆早期信息的能力渐进式下降——不是突然忘记，而是越来越模糊。
4. **成本非线性**：输入 token 成本随长度线性增长，加上更长的首 token 延迟，无节制的长上下文是不经济的。

Context Engineering 的核心假设是：**上下文窗口不是"装多少"的问题，而是"装什么、怎么装"的问题。**

> 前置知识请参考独立文档 《Context_Engineering_前置知识》，涵盖上下文窗口、Prompt Caching、KV Cache、Memory 等基础概念的通俗讲解。

📚 **关联报告**

- Prompt Engineering — Prompt 是"短输入"，Context 是"长输入"，两者是输入设计的连续谱

- 长上下文技术 — 上下文窗口"能装下"的技术基础

- LLM 应用框架 — LangChain/LlamaIndex 的上下文管理是应用层实践

三、纵向分析：从无意识到系统化

3.1 前 Context 时代：没有"上下文工程"之名，已有其实（2022–2024）

在 Context Engineering 被正式命名之前，它的核心组件已经在各个方向独立发展：

系统 Prompt 的进化（2022–2024）：2022 年末 ChatGPT 发布时，system prompt 只是 "You are a helpful assistant"——几十个 token，一个角色定义就够了。2023 年，开发者开始加入行为规则（"不要编造事实"）、输出格式（"用 Markdown 回答"）。2024 年，RAG 的普及让 system prompt 中加入了"如果检索到资料，基于资料回答；如果没有检索到，说不知道"等条件逻辑。system prompt 从几十个 token 增长到 500–2000 token。

"Lost in the Middle"（2023.07, arXiv:2307.03172） 是这个时期最重要的研究。Liu 等人发现了一个极其简单但影响深远的现象：给模型一堆文档，信息放在开头或结尾时，模型能准确提取；放在中间时，性能急剧下降。引用量超过 4000。

这个发现的意义超出了它本身——它让行业意识到**上下文不是"袋子"而是"舞台"**：同样内容放不同的位置效果天差地别。位置编排成为一门学问。

RAG 的实践积累（2023–2024）：社区在反复踩坑中发现——75% 的 RAG 问题来自数据准备和上下文质量，而非模型选择。什么 chunk 放进去、放几个、按什么顺序、要不要重复——这些"脏活"直接影响最终效果。但此时还没有人把这叫做"Context Engineering"。

对话历史管理的探索：LangChain 在这一时期推出了最早的一批 Memory 类——ConversationBufferMemory（全量保留）、ConversationSummaryMemory（用 LLM 压缩摘要）、ConversationBufferWindowMemory（滑动窗口）。开发者开始系统思考"过去的信息怎么保留"。

这个阶段的本质是：**各个组件各自为战——system prompt、RAG、memory、tool definitions——没有人把它们视为一个"上下文系统"来统一设计。**

3.2 基础设施铺设：Prompt Caching 与 MCP 协议（2024）

2024 年两个基础设施的出现，为 Context Engineering 的诞生提供了技术前提。

Prompt Caching 的推出：2024 年 8 月，Anthropic 率先为 Claude 3.5 Sonnet 和 Claude 3 Haiku 推出 prompt caching 公测。2024 年 10 月，OpenAI 跟进。机制的核心直觉很简单：如果每次请求都有大量相同的静态内容（system prompt、工具定义、few-shot 示例），为什么不让模型记住它们而不是重新计算？

效果惊人的好——静态内容部分的输入 token 成本降低 90%，延迟降低 80% 以上，首 token 延迟从秒级降到毫秒级。

Prompt Caching 的深层影响不是技术性的，而是**经济性的**。在 caching 之前，prompt 设计的第一原则是"最小化输入 token"。现在，大量静态知识（完整的 few-shot 示例集、详细工具文档、长篇幅的角色背景）可以低成本地放入上下文——开发者第一次被鼓励"写更多"而不是"写更少"。

MCP 协议（Model Context Protocol）：2024 年 11 月，Anthropic 发布 MCP 作为开放标准。它的定位是"AI 应用的 USB-C 接口"——让 Agent 与外部数据源之间的连接标准化。

MCP 的出现标志着 context 从"手工编排"走向"协议化"。开发者不再需要为每个工具写定制的 context 集成，而是通过 MCP 服务器统一管理工具定义和数据源的注入。

到 2026 年，MCP 被 ThoughtWorks 评为 2025 年最具影响力的 AI 基础设施之一，Codex、Claude Code 等主流开发工具都已支持。

3.3 概念诞生：一个 CEO 的推文点燃了火（2025.06）

Context Engineering 作为一个命名概念的诞生，故事线惊人地集中——前后不到一周：

2025 年 6 月 19 日，Shopify CEO Tobi Lutke 在 X 上发了一条推文，首次提出 "Context Engineering"：*"the art of providing all the context for the task to be plausibly solvable by the LLM."*

2025 年 6 月 23 日，LangChain CEO Harrison Chase 发表博客 "The rise of context engineering"，给出了更工程化的定义：*"building dynamic systems to provide the right information and tools in the right format such that the LLM can plausibly accomplish the task."*

2025 年 6 月 25 日，Andrej Karpathy 用一个比喻让这个概念爆发式传播：

> LLM = CPU（处理器），Context Window = RAM（工作内存），Context Engineering = Operating System（操作系统）

这个比喻之所以有效，是因为它让 AI 工程师立刻理解了概念的本质等级：**写 prompt 是写代码，context engineering 是设计操作系统**——两件完全不同量级的事。

2025 年 7 月 17 日，第一篇学术综述出现——Mei 等人的 "A Survey of Context Engineering for Large Language Models"（arXiv:2507.13334, 165 页, 62 引用）。学术界的快速跟进说明这个方向不是一时热点，而是行业共识已经形成。

3.4 迅速收敛：从概念到框架（2025 H2-2026）

概念诞生后的半年，Context Engineering 经历了从"是什么"到"怎么做"的快速收敛。

LangChain 的四策略框架（2025.07）：LangChain 在博客中将 context engineering 策略归纳为四类——Write（写入）、Select（选择）、Compress（压缩）、Isolate（隔离）。这是第一个被广泛引用的分类框架。

Anthropic 官方工程博客（2025.09）：Anthropic 发布 "Effective context engineering for AI agents"，系统阐述了 attention budget 理论和 context rot 概念。这是目前最权威的工程实践指南——来自最接近模型本身的团队。

Agentic Context Engineering（2025.10, arXiv:2510.04618）：Zhang 等人提出 ACE 框架——让 LLM 自主优化自己的上下文。这不是"人设计上下文给模型"，而是"模型设计自己的上下文"。ACE 被 ICLR 2026 接收，118 引用。

ACON 与 KV Cache 压缩（2026）：2026 年初，Agent 上下文优化（ACON）在 OpenReview 发表，实现 26–54% 的 Agent 上下文缩减。同期，语义 KV Cache 压缩的论文密集出现——TurboQuant（Google）、DeltaKV、ShadowKV，将 KV cache 压缩到 2–3% 的原始大小。

2026 年 AAAI 会议发表论文 "Algorithms for Context Engineering in LLM Inference"——这是顶级 AI 会议首次以 Context Engineering 为主题。

到 2026 年 4 月，Context Engineering 已经从一个推文概念变成了有论文、有框架、有工具的实践领域。Manning 出版社甚至出版了同名专著（Boni Garcia, 2026）。

3.5 对话历史管理的四条技术路线

对话历史管理是 Context Engineering 中最日常也最关键的组成部分。从 2022 到 2026 年，四条技术路线在实践中形成：

路线	核心做法	代表实现	适用场景
全量保留（滑动窗口）	保留最近 N 轮，丢弃旧的	ConversationBufferMemory	短对话
摘要压缩	LLM 定期把旧对话总结成几句话	ConversationSummaryMemory	长对话，需要保留关键信息
混合策略	近几轮保留原文，远的用摘要	SummaryBufferMemory	中长对话的平衡方案
向量检索	全部历史存入向量库，按需检索	VectorStoreRetrieverMemory	超长对话，需要精确回忆旧细节

四、横向分析：竞争图谱

4.1 场景判断

Context Engineering 作为一个新兴领域（~10 个月年轻），尚未形成成熟的"市场分类"。但它的核心组件分别在各自方向上存在充分竞争：

- **Prompt Caching**：三家巨头分庭抗礼（场景 B）
- **上下文压缩**：学术方法丰富，工程落地有限（场景 A–B）
- **框架支持**：LangChain 和 Anthropic 各有主张（场景 B）
- **安全/协议**：MCP 一家独大（场景 A）

4.2 Prompt Caching 三家对比

维度	Anthropic	OpenAI	Google Gemini
推出时间	2024.08 / 2025.08 GA	2024.10	2024
启用方式	手动标记断点（自动模式可用）	全自动，零代码改动	隐式自动 + 显式 API
最小缓存 token	1024–4096（因模型而异）	1024	2048
缓存深度	最多 4 个断点，content block 级	精确前缀匹配	前缀匹配
成本节省	Cache Read 省 90%	省 up to 90%	省 90% / 75%
TTL	5min（默认）/ 1h（付费）	in-memory 5–10min / extended 24h	默认 60min
额外费用	Cache Write +25% / +100%	无额外费用	显式缓存有存储费
控制力	精细（断点位置）	自动（控制力弱）	中（显式缓存 API）

Anthropic 的缓存策略最精细——手动标记断点允许开发者精确控制缓存边界。Cache Write 有额外费用（+25% 或 +100%），但 Cache Read 低至 10% 的输入价格。在多轮对话场景中，断点自动向前移动的 eesign 减少了人工干预。

OpenAI 的缓存与 Anthropic 走不同路线——全自动，零配置。开发者不需要学任何新概念就能享受缓存收益。代价是控制力弱——`prompt_cache_key` 参数文档透明度不足。更适合"不想操心"的团队。

Google 的缓存对多模态场景（视频、音频）支持最好。隐式缓存对所有项目默认启用，显式缓存提供更高控制力。最大缓存 10MB。多模态内容的上下文工程在公司内部是一个独特优势。

4.3 上下文压缩技术对比

方法	发布时间	核心思想	压缩率	落地情况
LLMLingua	2023 (EMNLP)	小模型识别并删除非必要 token	up to 20x	已集成 LangChain/LlamaIndex
Selective Context	2023 (EMNLP)	识别和剪枝输入冗余	~2x	论文阶段
LongLLMLingua	2024 (ACL)	长上下文针对性优化	up to 4x	同上
LLMLingua-2	2024	BERT encoder 做 token 分类	task-agnostic	同上，速度 3-6x 提升
StreamingLLM	2023 (ICLR 2024)	Attention Sink + window attention	无限长	已集成推理框架
KVQuant	2024	KV cache 量化	内存减少 4x	工程落地中
TurboQuant	2026 (Google)	3-bit KV cache 量化	内存减少 ~10x	论文阶段
DeltaKV	2026	只存储 KV 残差	重构≤10% tokens	论文阶段

LLMLingua 系列（Microsoft Research, GitHub 6.1k stars）是唯一大规模工程落地的压缩技术。`coarse-to-fine` 策略——先用预算控制器分配全局 token 预算，再用 token 级迭代压缩——在不牺牲太多精度的前提下实现 2-20x 压缩。LongLLMLingua 在 RAG 场景下额外提升 21.4% 的性能——压缩不仅没伤害效果，反而因为"挤掉了噪音"改善了输出。

StreamingLLM 在另一个方向上做出突破。MIT 和 Meta 的研究者发现了 Attention Sink 现象：初始 token 即使语义无关也会获得极高的注意力分数。利用这个特性，保留初始 token + 最近窗口的 KV cache，模型可以处理**无限长度**序列而无需微调。对比纯滑动窗口方案加速 22.2x。

但压缩技术的工程落地整体偏慢——LLMLingua 是唯一大规模集成的主流方案。论文中的压缩率在真实场景中往往达不到，且压缩后的结果验证缺乏标准化评估方法。

4.4 框架对 Context Engineering 的支持

框架	内存管理	上下文压缩	Caching 支持	协议支持
LangChain	5 种 Memory 类	有限（LLMLingua 集成）	通过 API 参数	MCP 客户端
LlamaIndex	ChatMemoryBuffer	有限	通过 API 参数	有限
DSPy	History 签名字段	编译器自动优化	不直接支持	有限
SGLang	RadixAttention（KV cache 复用）	内建前缀缓存	推理层内置	有限

LangChain 对 Context Engineering 的支持最全面——从 2023 年的 Memory 类到 2025 年的四策略框架，再到 Deep Agents 的自动上下文压缩。这在 2026 年是做 Context Engineering 的首选框架。

SGLang 的 RadixAttention 走了一条完全不同的路——不是在应用层做 context 管理，而是在推理引擎层用基数树自动管理 KV Cache 复用。few-shot 示例共享前缀、多轮对话的历史、self-consistency 的多次采样——这些常见的 context 复用模式被自动处理，吞吐量提升最高 5x。

4.5 社区口碑与行业影响

Hacker News 和 Reddit 上的讨论（2026）：

- **Context Engineering vs Prompt Engineering** 是最热的争论。"Prompt engineering is about what to say, context engineering is about what the model knows"——这个区分被广泛接受。
- **对 Prompt Caching 的真实反馈**：正面集中在"省了 90% 成本"，负面集中在"最小 token 阈值容易被忽视"和"Anthropic 的 cache write 费用有时超过省下的 cache read"。
- **"两大阵营"**：坚持"长上下文直接出"的信任派 vs 坚持"精心编排"的成本控制派。后者的核心论据是 token 成本——不是技术上做不到，是经济上不值得。
- **SGLang 在 LLM serving 社区的广泛认可**：RadixAttention 被评价为"LLM 基础设施层的 context engineering"。

行业报告数据：实施结构化 Context Engineering 的组织报告 3x 更快的 AI 部署速度（来自 LangChain 2025 年行业数据）。

角色演变：从"Prompt Engineer"（2023 H1）到"AI Engineer"（2024）到"Context Architect"（2025-2026）——角色的定义在被 Context Engineering 重新塑造。

五、横纵交汇洞察

5.1 一条推文定义了赛道：Context Engineering 的"意外诞生"

纵向历史中最有趣的现象是：**Context Engineering 不是学术界提出的，不是研究实验室发现的，而是一个 CEO 在 X 上发了一条推文。**

Tobi Lutke 在 2025 年 6 月 19 日的原始 tweet 不到 30 个词，也没有引用任何论文。但它命中了行业正在经历却无法表述的痛点——每一个做 AI Agent 的团队都在凌晨三点调试 context 溢出、工具定义被截断、历史对话丢失。他们需要这个词来命名正在做的事。

Karpathy 的 CPU/RAM 类比之所以迅速传播，不是因为它学术正确，而是因为它让每个工程师瞬间理解了这个概念的战略高度。**这是个大号比喻胜过十篇论文的罕见案例。**

LangChain CEO Harrison Chase 在 4 天后的跟进，把行业最热门的框架绑定了这个概念——LangChain 从此不仅是"prompt 编排框架"，还是"context engineering 平台"。商业利益完美契合概念推广。

这个诞生故事本身就是一个洞察：**当行业已经准备好接受一个概念时，概念的提出者是谁已经不太重要——重要的是谁第一个用对了词。**

5.2 Prompt Caching：一个被低估的"经济革命"

Context Engineering 快速崛起的最大推手不是技术突破，而是成本结构的变化。

在 prompt caching 之前，每增加 1000 token 的 system prompt = 每次调用多 1000 token 的成本。开发者的自然反应是——压缩 system prompt 到最小。这跟 prompt 质量直接冲突。

Caching 出现后，这个等式变了：前几千 token 的 system prompt 只需要付一次 cache write 费用，后续每次调用都走 cache read（10% 的价格）。**成本从线性增长变成了近乎固定。**

这个变化的经济效应是：开发者在 prompt 设计中从"少写"变成了"多写"——写更详细的角色定义、更完整的工具描述、更多的 few-shot 示例。这不是 prompt 设计的进步，是**预算约束的松绑**。

横向对比中，三家巨头对 caching 的定价策略体现出不同的优先级：

- **Anthropic**：精细化控制，但收 cache write 费用。适合规模够大、cache hit 率高的场景。
- **OpenAI**：零配置零额外费用。适合中小规模、不想操心的团队。
- **Google**：面向多模态场景。适合视频/音频分析等媒体密集型应用。

选择哪家不取决于"谁的缓存技术最好"，而取决于**你的 context 中有多少静态内容、有没有能力做断点优化**。

5.3 一个还在"做加法"的领域

跟 RAG 或 Prompt Engineering 不同，Context Engineering 作为一个领域还非常年轻（约 10 个月）。纵向分析中可以看到一个清晰的特征：**这个领域还在"做加法"——不断有新概念加入、新论文涌现、新工具出现，还没有完整的收敛。**

对比来看，RAG 在 2023 年经历了从 Naive RAG 到 Advanced RAG 再到 Modular RAG 的阶段收敛。Prompt Engineering 在 2023-2024 年经历了"Is it dead"争论后收敛到工程化。Context Engineering 在 2026 年还没有一个公认的分类框架——LangChain 的四策略只是其中一种。

由此，几个事情变得清晰：

- **现在入场的组织有先发优势。** 框架未定意味着最佳实践尚未固化，动手早的团队积累的经验就是下一轮的护城河。
- **工具生态还在早期。** 没有"Context Engineering 平台"这个产品品类——PromptLayer 做版管理但不做 cache 优化，Helicone 做可观测但不做压缩。这是明显的市场空白。
- **标准化的需求在积累。** 如果 MCP 继续普及，可能会出现类似"上下文窗口预算配置文件"的标准格式。

5.4 Context Engineering 和 Prompt Engineering 的分工

横向对比中最容易被误解的关系是 Context Engineering 和 Prompt Engineering 的关系。

一个简单的分界：**Prompt Engineering 研究的是"怎么写"，Context Engineering 研究的是"写什么"。**

	Prompt Engineering	Context Engineering
核心技能	语言表达、推理策略	信息架构、系统设计
关注对象	指令文本本身	全部上下文（指令+数据+工具+记忆）
优化目标	让模型理解意图	让模型拥有足够的信息
典型产出	措辞、CoT 模板	缓存策略、token 预算分配、压缩方案
可自动化程度	高（DSPy 等框架）	中（部分可自动化，部分需人类判断）

关系：**Context Engineering 不是替代 Prompt Engineering，而是把它包含进去了。** 好的 context engineering 一定包含好的 prompt 设计，但反过来不成立——你可以在 prompt 写得很好但 context 管理一塌糊涂。

5.5 三个未来剧本

剧本一：Context Engineering 融入 AI 应用开发全流程（概率 65%）

3–5 年内，"设计上下文"成为 AI 应用开发的标准环节——每个 AI 工程师都在做 Context Engineering，就像每个后端工程师都在做数据库查询优化。工具链成熟化：开源的 context 配置格式出现，可视化 context 设计工具普及，自动 context 优化成为推理引擎的内建功能。SGLang 的 RadixAttention 路线（推理层自动优化）可能成为终局方案——开发者不需要手动管理几乎全部上下文，引擎帮你做了。

剧本二：Long CoT + Ultra-long Context 让大部分 Context Engineering 自动化（概率 25%）

如果模型的推理能力继续增强（o1 → o2 → o3），并且上下文窗口扩展到 10M+ token 的同时首 token 延迟降到 100ms 以下，大量手动的 context 编排工作会变得不那么必要。模型的内部推理能力可以直接"看完"整个 context 并找出相关信息，就像人类翻完一本书找出关键段落。此时 Context Engineering 退化为"把可能相关的资料全部丢进去，让模型自己处理"。

剧本三：安全合规要求让 Context Engineering 变成强制规范（概率 10%）

如果 prompt injection 和数据泄露问题在 Agent 系统的大规模部署中引发实际的安全事件，监管介入可能要求所有生产级 AI 系统实施标准化的 context 隔离和权限控制。此时 Context Engineering 从"最优实践"变成"合规要求"——你不仅要设计好 context，还要证明你的 context 是隔离的、可审计的、有访问控制的。MCP 协议的采用可能从"好用"变成"必须"。

5.6 第一性原理追问

Context Engineering 的终极问题是：**AI 应用的"上下文"应该由人设计，还是让模型自己决定?**

ACE 框架（2025.10）和 SGLang 的路线都指向"交给模型/引擎"。LangChain 和 Anthropic 的路线倾向于"人设计系统，模型执行"。

2026 年的现实是两者都在探索。但可以预见的终局是：

"人设定架构和约束，模型/引擎自动优化填充。"

就像现代数据库——你定义 schema 和索引策略，但查询优化器自动决定怎么执行查询。Context Engineering 的未来可能不是"人设计每一条 context"，而是"人定义 context 的规则和优先级，系统自动编排"。

但如果那样——Context Engineering 会不会变成另一个"中间层"？像 Q-Former 一样，在模型能力和人类需求之间做一个短暂的桥梁，然后随着模型能力的提升而被溶解？这个问题没有答案，但它指向了 Context Engineering 可能是一个**过渡性学科**——不是永久性的新工种，而是从"手动 prompt"到"完全自动化"之间的必经阶段。

六、信息来源

核心论文与文章

论文/文章	来源	访问时间
Lost in the Middle (Liu et al.)	arxiv.org/abs/2307.03172	2026-04-30
LLMLingua	arxiv.org/abs/2310.05737	2026-04-30
StreamingLLM	arxiv.org/abs/2309.17453	2026-04-30
Selective Context	arxiv.org/abs/2310.06201	2026-04-30
Context Engineering Survey (Mei et al.)	arxiv.org/abs/2507.13334	2026-04-30
Agentic Context Engineering (ACE)	arxiv.org/abs/2510.04618	2026-04-30
Tobi Lutke 首次提出 Context Engineering	x.com/tobi (2025-06-19)	2026-04-30
Harrison Chase: The rise of context engineering	blog.langchain.dev (2025-06-23)	2026-04-30
Anthropic: Effective context engineering	docs.anthropic.com (2025-09)	2026-04-30
Redis: Context engineering best practices	redis.io (2025-09-26)	2026-04-30
Manning: Context Engineering 专著	manning.com (2026)	2026-04-30

产品与技术来源

来源	URL	访问时间
Anthropic Prompt Caching 文档	docs.anthropic.com	2026-04-30
OpenAI Prompt Caching 文档	platform.openai.com	2026-04-30
Google Context Caching 文档	cloud.google.com/vertex-ai	2026-04-30
LangChain Memory 文档	docs.langchain.com	2026-04-30
SGLang RadixAttention	github.com/sgl-project/sglang	2026-04-30
DSPy 文档	dspy.ai	2026-04-30
MCP 协议规范	modelcontextprotocol.io	2026-04-30
Comet Opik	github.com/comet-ml/opik	2026-04-30
ByteByteGo: Context Engineering Guide	bytebytego.com (2026-04)	2026-04-30
ThoughtWorks: MCP 年度技术雷达	thoughtworks.com	2026-04-30
Hacker News (context engineering 讨论)	news.ycombinator.com	2026-04-30
Reddit r/LocalLLaMA (MoE 讨论)	reddit.com/r/LocalLLaMA	2026-04-30
Wikipedia Prompt Engineering 词条	wikipedia.org	2026-04-30

本文是横纵分析系列的第 14 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法，融合语言学历时-共时分析、社会科学纵向-横截面研究设计、以及竞争战略分析的核心思想。